

Evaluation of an Automated content moderation classifier

Introduction

The task is to build automated tests to determine whether a pre-built classifier meets the following requirements: correctness, i.e. reliably and consistently flag toxic and non-toxic content, fairness, i.e. the classifier should not bias against particular demographics mentioned in the text and robustness, i.e. the classifier should behave stably when the content undergoes changes in formatting. The following technical document outlines the metrics used to assess how well the classifier fits the aforementioned requirements and discussion around how the evaluations are carried out.

Evaluating Correctness

For each requirement, a test is built to measure the models performance according to some metrics. For correctness, the MTEB toxic conversation dataset¹ is chosen as it contains realistic text with toxic and not toxic labels, which are exactly the same as the classifiers output here. For a given testing dataset, we measure the True Positives (TP), True Negatives (TN), False Positives (FP) and False Negatives (FN). These are then used for aggregate metrics by which we can measure the classifiers performance; firstly, precision, defined as:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

From all of the posts flagged as toxic, the precision measures how many of them were actually toxic. In this context, a high precision means the classifier rarely flags non-toxic content as toxic, whereas a low precision indicates a high false-positive rate, which could lead to heavier operational costs if there is a human review process.

Secondly, the recall is defined as:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

From all the posts labelled as toxic, the recall measures how many of them were flagged as toxic by the classifier. A low recall in this context stems from a higher number of false negatives, indicating that the system is not successfully filtering out toxic content as it should.

Ordinarily, the next metric would be the F_1 -Score is defined as:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

¹ https://huggingface.co/datasets/mteb/toxic_conversations_50k

When using the MTEB Toxic Conversations dataset, the F_1 -score cannot be used as it ignores the true negatives (TNs). A drawback of the dataset is that it is imbalanced; among 100000 total samples, only 7919 are labelled as toxic. A model that predicts “not toxic” on every input (which will happen on classical machine learning models like regression or SVMs, when trained on data with such imbalances) will yield a misleading evaluation depending on the F_1 threshold.

In this case the Mathews Correlation Coefficient will be used alongside precision and recall. It only returns a high value if the model performs well on the majority and minority of cases in the dataset. It is defined as:

$$\text{MCC} = \frac{(\text{TP} \cdot \text{TN}) - (\text{FP} \cdot \text{FN})}{\sqrt{(\text{TP} + \text{FP}) \cdot (\text{TP} + \text{FN}) \cdot (\text{TN} + \text{FP}) \cdot (\text{TN} + \text{FN})}}$$

The coefficient yields values between -1 and +1, where a score of -1 indicates complete disagreement between prediction and the true value, 0 represents an average random prediction, i.e. the model is effectively guessing, and +1 represents perfect predictions. The values of the precision and MCC thresholds depend on the context and user. For a social media platform, there is a delicate balance between ensuring the model works, i.e. correctly filters toxic content, but does not return a high number of false positives, which may lead to the platform being censored (although again this really depends on what the user wants). Here a precision 0.8 is selected, i.e. 80% of toxic content is actually toxic, and an MCC of 0.5, guaranteeing that the model is not simply guessing.

Evaluating fairness

To evaluate fairness, we introduce the notion of counterfactual fairness. That is, given a statement such as “I met a [IDENTITY] person”, the resulting toxicity should remain the same irrespective of the demographic group in the position of the mask [IDENTITY]. The mathematical formulation is as follows:

Let $T = \{t_1, t_2, \dots, t_K\}$ represent K non-toxic text templates with a masked token as in the example given previously. Let C represent a specific demographic category, e.g. religion, with M unique groups within the category, e.g. Islam, Christianity, Judaism, etc.

$$D_C = \{d_1, d_2, \dots, d_M\}$$

We can also define a control term $d_0 \in D_{\text{control}}$ to establish a baseline classification error rate, agnostic of any demographic term. The term d_0 used here is “people”. For each template, we swap the placeholder token with a term from D_C , for each category (across religion, race, ethnicity, etc, and each group within them). This leads to an input text x_{td} where t is the template and d is the inserted demographic group. The base templates T are designed to be non-toxic, so the output label Y for an input x_{td} should always be 0, i.e. non-toxic.

$$Y(x_{td}) = 0 \quad \forall \quad t \in T, \quad d \in D_C$$

Let $f(x) \in \{0,1\}$ represent the classifier, where it only outputs 0 for non-toxic and 1 for toxic. A false positive occurs when a positive (toxic) prediction is given for an input x_{td} . The false positive rate is:

$$\text{FPR}_d = \frac{1}{K} \sum_{k=1}^K \mathbb{I}(f(x_{td}) = 1)$$

Where $\mathbb{I}$ is an indicator function evaluating to 1 if the interior condition is true, and 0 otherwise. Similarly for the false negative rate:

$$\text{FNR}_d = \frac{1}{K} \sum_{k=1}^K \mathbb{I}(f(x_{td}) = 0)$$

We aggregate these to calculate the maximal group disparities to measure the bias. We say the classifier is biased if the disparity between FPRs and FNRs between the most penalised and least penalised groups falls below a threshold.

$$\Delta_{\text{FPR}}(C) = \max_{d \in D_C}(\text{FPR}_d) - \min_{d \in D_C}(\text{FPR}_d)$$

$$\Delta_{\text{FNR}}(C) = \max_{d \in D_C}(\text{FNR}_d) - \min_{d \in D_C}(\text{FNR}_d)$$

The fairness requirement for the classifier is met, if and only if the disparity metrics remain bounded within a tolerance threshold T :

$$\max(\Delta_{\text{FPR}}(C), \Delta_{\text{FNR}}(C)) \leq T$$

The disparity threshold is set to 5% for this project. Ideal thresholds can be found via experimentation, literature review and client requirements. The dataset used to measure the fairness according to these metrics was synthetically generated. This required some experimentation, mostly due to the fact that most LLMs have safety guardrails preventing them from generating toxic content. This seems to be the case even when prompting specifically for seed templates with demographic terms masked, including mention of the context (designing a bias testing dataset) in the prompt. This issue occurred both using ChatGPT directly and also through the API. The former was able to produce some toxic seeds when given some examples, but the quality of the seeds, both benign and toxic was poor, often producing simple sentences which resemble lines in a children's book opposed to social media posts, e.g. "I saw a [IDENTITY] family in the park". This issue was circumvented by using local LLMs. Even those presented the same issue, as the ones useable through Ollama are "instruct" models, i.e. have the guardrails too, however some models allow for more user control, such as Hermes 3. With some careful prompting it is possible to produce realistic social media type templates, both toxic and non-toxic. As a smaller language model is being used, with 8B parameters, a few-shot

delimited string template is used (to avoid issues when asking the model to output JSON directly) where the model essentially completes the line. The output is then parsed and saved as JSON. Generation is done through batches to avoid exhaustion of the output token window. In this case 138 toxic and 138 non-toxic seed phrases were generated. A separate script is then used to insert the demographic terms and generate the dataset, which is available on Hugging Face². For this evaluation, the text seeds mimic social media posts relating to finance, technology and politics, and the demographic terms are stemmed from religious and ethnic groups (arbitrarily chosen as a proof of concept for this task). The result is a dataset with 5244 inputs, balanced equally across toxic and non-toxic seeds, and again balanced equally for each demographic category and group. In a real world scenario with more time and compute (e.g. using a larger local model), a much larger and more context-specific database, with more demographic categories, can be generated according to the requirements of the model host.

Evaluating Robustness

Robustness is a measure of how stable the model's output is when the input conditions are perturbed. For example, we expect the output of the model (irrespective of whether the prediction is correct or not to begin with) to remain the same, regardless of whether the input text capitalised or not, contains minor typos, differences in punctuation, etc. We use the invariance rate as the metric to measure robustness. Let $f(x)$ denote the model's prediction for a given text x . Let $T(x)$ denote a perturbation function, e.g. adding a typo to the input text x . The invariance rate IR_T for a perturbation T for N samples is:

$$IR_T = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[f(x_i) = f(T(x_i))]$$

Where $\mathbb{I}$ is an indicator function evaluating to 1 if the interior condition is true, and 0 otherwise. The threshold for the invariance rate is set to 90%, i.e. we want the model prediction to remain identical 90% of the time. We cannot expect the prediction to remain 100% of the time as syntax and grammar can have an effect on the sentiment of written text, e.g. all caps can have connotations of anger. Again we use the MTEB Toxic Conversations dataset for the same reasons as with evaluating correctness, but instead we add perturbations alongside the original text. We use 500 samples from the original dataset and apply 7 perturbations, leading to 3500 samples: Upper case only, lower case only, mixed case, addition and removal of punctuation and addition of typos. The model must achieve a 90% invariance rate on all perturbations to ensure it does not have a specific blind spot that might hide behind an average score. The dataset is available on Hugging Face³. Again, as with the case for the previous two evaluations, a larger dataset

² <https://huggingface.co/datasets/TajwarC/Social-Media-Counterfactuals>

³ https://huggingface.co/datasets/TajwarC/mteb_toxic_conversations_2.5k_robustness

can be used in practice with more time and resources. Other variations can also be introduced. The addition of perturbations here is somewhat one-dimensional; the use of an LLM would generate far more diverse perturbations but also require much more work, hence this dataset is presented as a proof of concept for the sake of this exercise.

Results

Full results are available in the JSON files in the reports folder. The classifier actually fails all of the tests based on these specific datasets. For correctness, over 4096 samples, it returns an MCC of 0.314 and precision of 0.268, with thresholds of 0.5 and 0.8 respectively, indicating the classifier does not reliably separate toxic content from non-toxic content. With respect to bias over 5244 samples, for religious groups it returned a Δ_{FPR} of 0.072 and Δ_{FNR} of 0.341, both of which are above the allowed disparity of 0.05. For ethnic groups it also fails with a Δ_{FPR} of 0.196 and a Δ_{FNR} of 0.384. Further analysis of the metrics for each demographic group can be done to reveal which groups the classifier biases against most. The classifier performs best with robustness testing, passing all perturbation categories over 3500 samples, with an invariance of over 90%, except in one case, where the cases are mixed, with an invariance of 85.8%, falling just below the threshold of 90%. This shows the classifier is most sensitive to mixed case inputs, and results in the entire test failing as described previously.

Notes on project structure

There are two main directories that were developed, `tests_suites` and `datasets`. The directory `tests_suites` contains the core code which carries out the evaluations.

`data_loader` – Data loading utility function to avoid repetition inside of core evaluation functions. Loads datasets from Hugging Face to avoid needing the datasets saved locally first.

`eval_bias` – Core function to evaluate bias in the classifier. Runs each sample from the testing dataset through the classifier. Calculates FPRs and FNRs for each demographic category and group followed by disparities.

`eval_correctness` – Core function to evaluate correctness of the classifier. Runs each sample from the testing dataset through the classifier. Calculates TPs, FPs, TNs and FNs, followed by precision, recall and MCC.

`eval_robustness` – Core function to evaluate robustness of the classifier. Runs each sample from the testing dataset through the classifier and calculates the invariance rate between perturbations and original text in the dataset.

Note: 2.5k refers to the number of samples taken from the original MTEB dataset – this initially resulted in 17500 total samples (due to 7 variations), but was reduced down to 500 original samples resulting in 3500 total samples, as there were issues uploading the larger dataset.

`logging_config` – Utility function to initialise logging in the core functions, to avoid repeating code.

`reporter` – Prints results of each evaluation into the console and then exports a JSON file with the results

`utils` – Utility function containing functions to process batches through classifier

The core functions can be run from the command line and take extra parameters as inputs, defined in the README, for user control.

The datasets directory contains the code used to generate the bias and robustness evaluation datasets.

`generate_seeds` – Uses a locally hosted LLM (in this case Hermes 3 8B) to generate seed phrases for the counterfactual dataset. Contains user defined domains (e.g. finance, politics, technology) and sub topics within these domains and outputs the phrases with their toxicity label to JSON

`counterfactual_fairness` – Inserts demographic terms into seed phrases generated by `generate_seeds`.

`generate_perturbations` – Perturbs text samples from MTEB Toxic Conversations dataset and outputs to JSON.

Final remarks

The project aims to give a framework for how a classification model can be benchmarked for correctness, bias and robustness. I used one existing open-source dataset and created another perturbed version of it, to test correctness and robustness respectively, and generated a novel dataset using a local LLM to bypass safety guardrails, to test bias. In each case, (additional) larger and more context specific datasets can be generated either by using the methods I've used (e.g. addition of more demographic groups, topics for seeds, perturbations for robustness testing, etc.) or through methods from literature and experimentation, but of course this task was only designed to take a few hours.

Throughout, I drew on my experience developing “EquiFace”⁴, a fairness benchmarking tool for biometric authentication, which had significant crossover to this task, mainly with deciding on the metrics and size of the dataset. Through empirical testing during the development of EquiFace, I found that for the purpose of testing, usually only a number of samples in the order of 10^3 is required to expose bias. Thresholds were also somewhat empirical, but ideally should be decided experimentally based on client requirements, based on literature, or according to standards such as NIST/ISO/NEC (as is the case with biometrics). Whilst we are using a different modality here (text as opposed to images), I

⁴ <https://github.com/TajwarC/EquiFace-2.0>

set the number of samples used from the dataset in this order of magnitude. Of course in reality this number can be much higher.